ELEC-H-310
Th05: Asynchronous sequential circuits

Dragomir Milojevic
Université libre de Bruxelles

v3.1

Today

1. Race conditions
2. Methods to solve race conditions
3. Example of asynchronous implementation
4. Synthesis of Moore & Mealy machines including output
5. Synthesis of Moore & Mealy machines

1. Race conditions

State table encoding

- Let's start with an example of an optimized state table
  - ▹ How do we know that this state table is optimized?
- Les's use the default state encoding:
  `1` $\rightarrow$ `00`, `2` $\rightarrow$ `01`, `3` $\rightarrow$ `11`, `4` $\rightarrow$ `10`
- We can then derive optimized & encoded state table

	ab			
	00	01	11	10
1	**1**	2	3	2
2	1	**2**	4	**2**
3	1	**3**	**3**	4
4	1	3	**4**	**4**

	ab			
Y_2Y_1	00	01	11	10
00	**00**	01	11	01
01	00	**01**	10	**01**
11	00	**11**	**11**	10
10	00	11	**10**	**10**

y_2y_1

State equations & logic optimization (K-Maps)

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	11	01
01	00	**01**	10	**01**
11	00	**11**	**11**	10
10	00	11	**10**	**10**

y_2y_1 \ ab	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	1	1	1
10	0	1	1	1

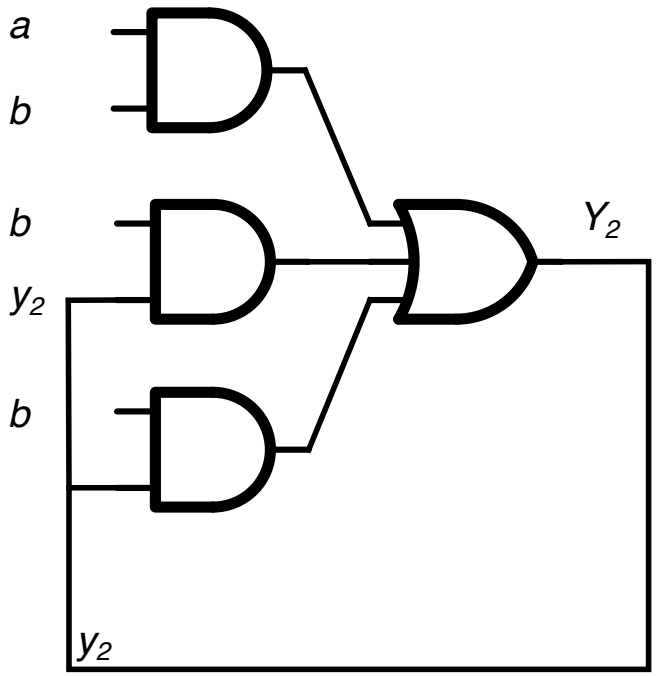
$$Y_2 = ab + y_2b + y_2a$$

y_2y_1 \ ab	00	01	11	10
00	0	1	1	1
01	0	1	0	1
11	0	1	1	0
10	0	1	0	0

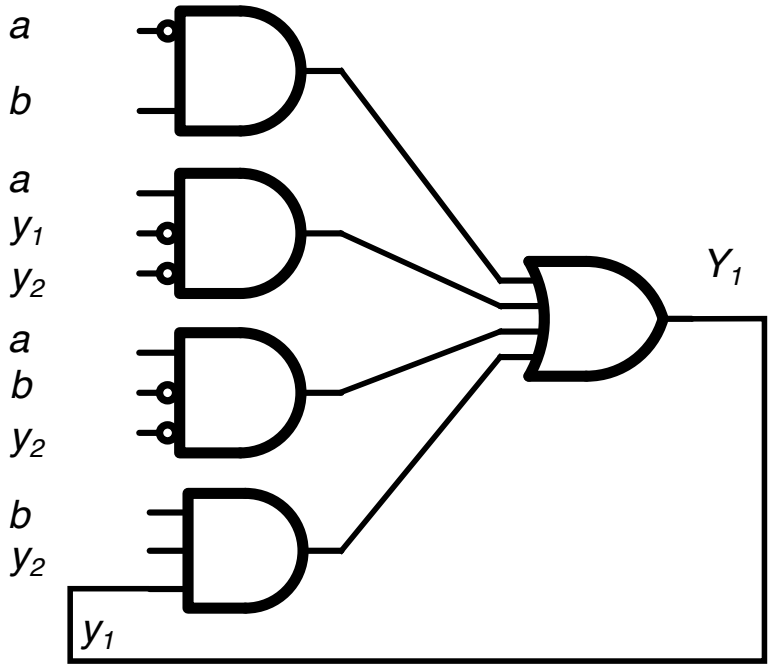
$$Y_1 = a'b + y_2'y_1'a + ab'y_2' + y_2y_1b$$

Let's build the logic circuit out state equations

Two expressions result in two independent circuits with feedback loops to allow predicted future to become present: $y_i \leftarrow Y_i$



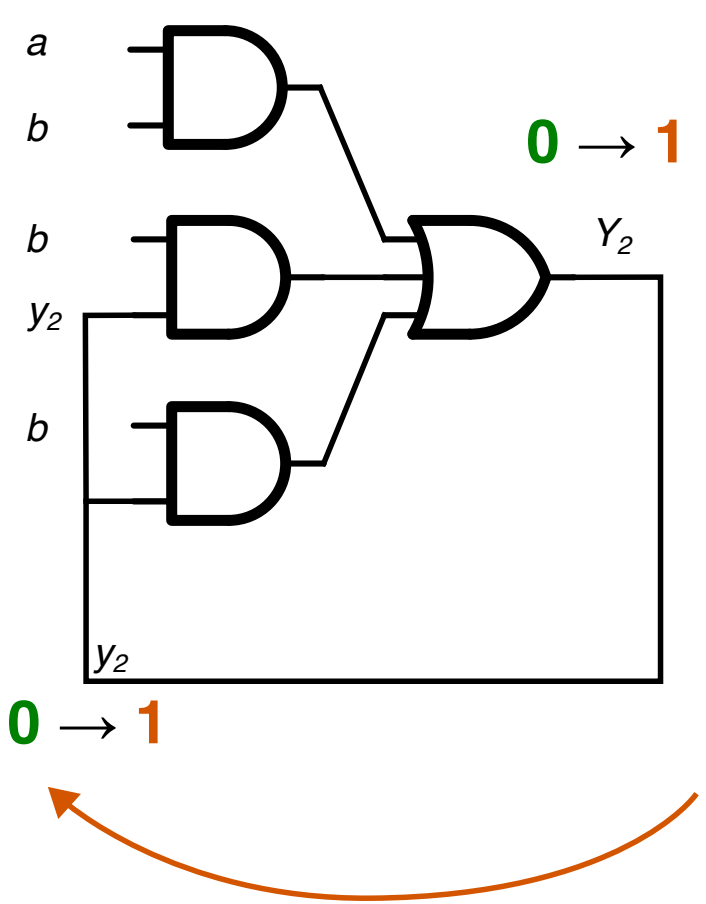
$$Y_2 = ab + y_2 b + y_2 a$$

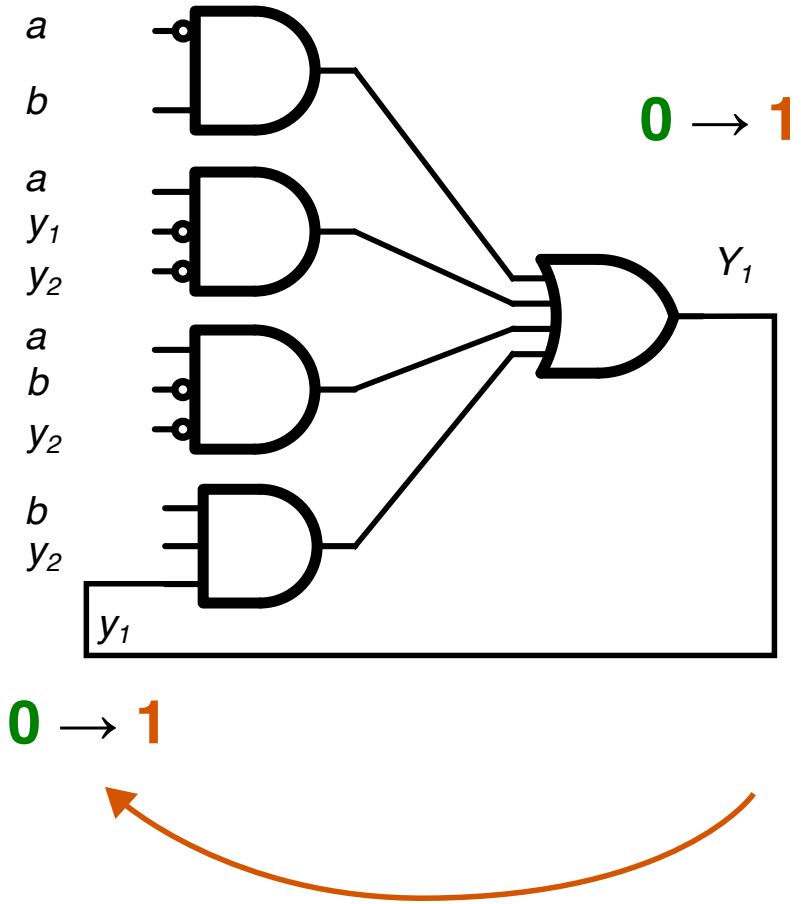


$$Y_1 = a'b + y_2' y_1' a + ab' y_2' + y_2 y_1 b$$

Race conditions at circuit level

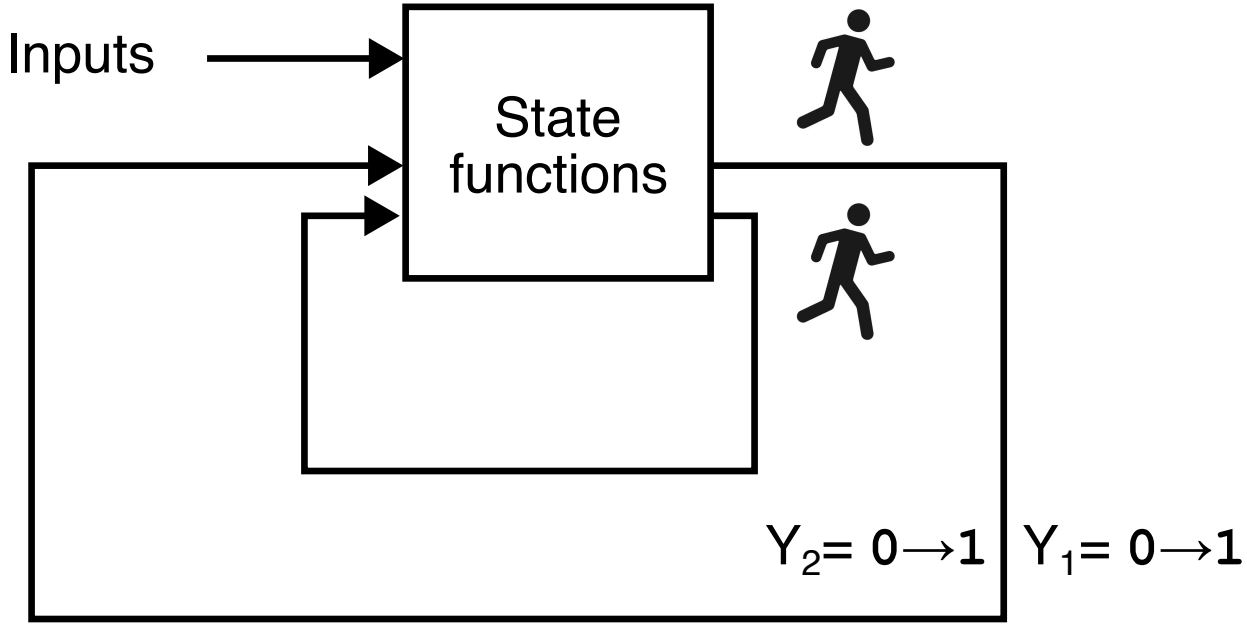
During the update of the state variables, when predicted, future becomes present ($y_i \leftarrow Y_i$) it is possible that two (or more) state variables switch from '0' to '1' at the same time





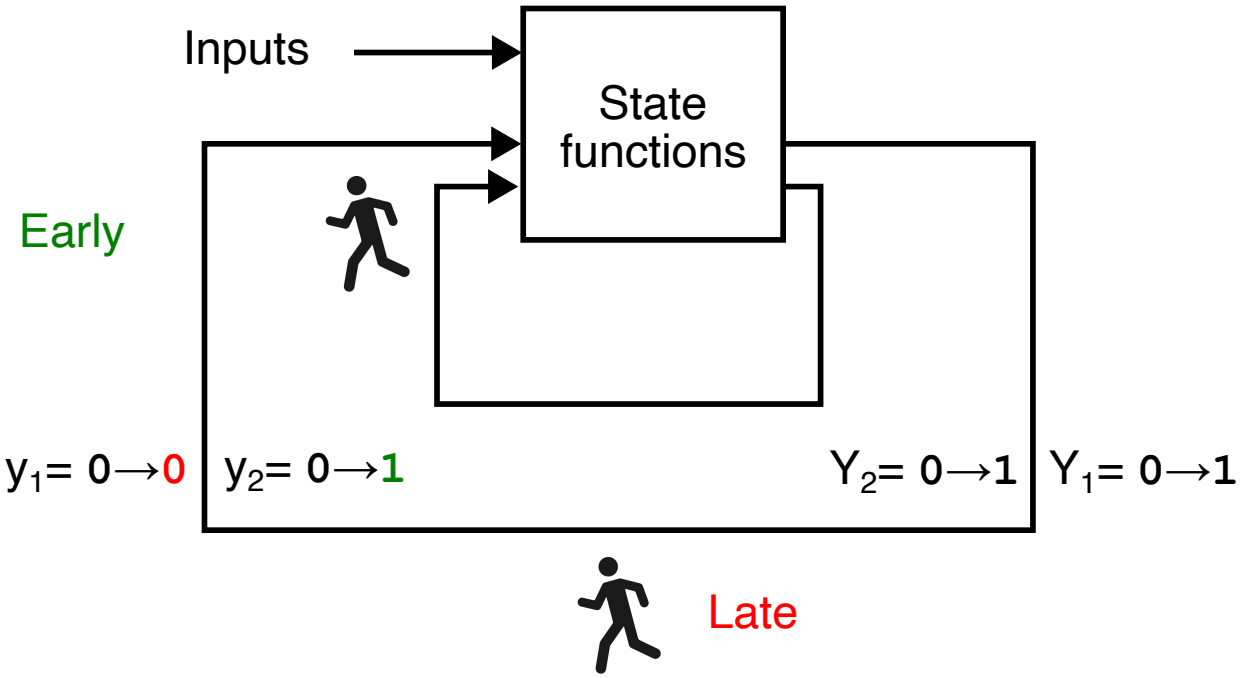
Race conditions (analogy) explained (1/2)

- Our circuit has 2 state variables that we see as two runners
- When input changes, internal logic will compute the future state
- If the current/future state should be the transition from `00` to `11`, both runners should start their "race" at the same time



Race conditions (analogy) explained (2/2)

- Two "running lanes" will not have the same length (wire), speed of two runners will not be the same (transmission speed) etc.
- For sure one of the two runners will arrive before the other
- If, instead of `11`, the new destination, as seen by state function logic, is `01` (or `10`), the system may end up in another state



There are race conditions

Race conditions: back to the state table

- System is in stable state **00**; inputs *ab* change from `00` to `11`, future state should become `11` (transition) to reach stable destination state **11**
- But due to race conditions instead of `11` system could see `10` and in our example `10` is stable, the system will stay there! Note that the same will occur if races send us to `01` instead of `10`
- Due to race conditions destination state is altered!

ab

Y_2Y_1	00	01	11	10
00	**00**	01	11	01
01	00	**01**	10	01
11	00	**11**	**11**	10
10	00	11	**10**	**10**

y_2y_1

THIS IS BAD, we must avoid this!!!

Race conditions & solutions

- Definition – race conditions will occur with instantaneous change of 2 or more state variables from source to destination stable state
- In sequential logic circuits it is mandatory that we don't have race conditions to ensure deterministic behaviour of the automata
- There are two different ways to solve the race conditions problem, resulting in two (VERY!) different logic circuit classes:
  - Asynchronous logic circuits – we make sure there are no races during state transitions, i.e. any state transition will have up to 1 internal variable change at most; covered in this chapter
  - Synchronous logic circuits – we use memory to synchronize state values; any transient values on the input of these memories will be "filtered out" (ignored); just think of a D Flip-Flop; we may have races in the state tables, but they are now harmless; this is what we do in the next lecture

Race conditions in state tables

- In the left state table below race conditions are marked in red $\rightarrow$ Hamming Distance between present and future is > 1
- But don't mix things: inputs can change instantaneously, they are random variables and as such can change in any way
- In the right table inputs can change from 00 to 11 $\rightarrow$ these are not race conditions[1]

Y_2Y_1 \ ab	00	01	11	10
00	**00**		11	
01		**01**		10
11	00	**11**	**11**	
10		01	**10**	**10**

y_2y_1

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	11	01
01	00	**01**	10	01
11	00	01	**11**	10
10	00	11	**10**	**10**

y_2y_1

[1] True for other input changes too: e.g., 01 to 10, 11 to 00, etc.

2. Methods to solve race conditions

State table encoding

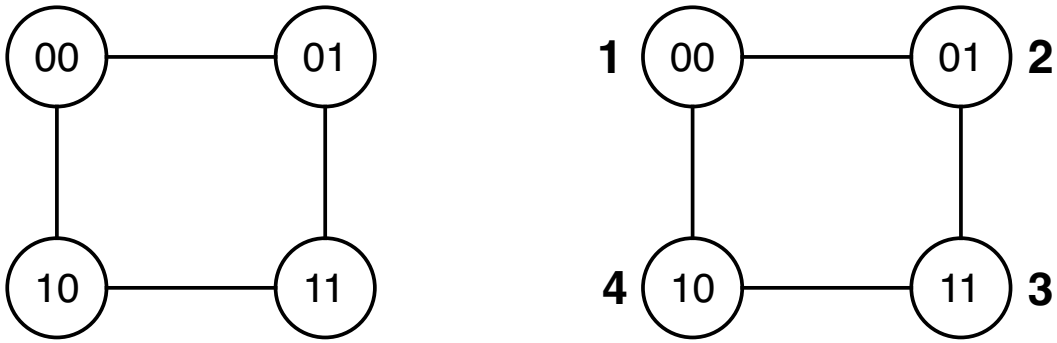
- We will cover three different methods that will allow us to implement operational sequential asynchronous circuits:
  - METHOD1, METHOD2 and METHOD3
- To ensure optimal physical implementation (logic complexity) these methods are applied in a **given order**:
  - METHOD2 is applied only if METHOD1 can't provide a workable solution
  - METHOD3 applied only if METHOD1 and METHOD2 can't provide a workable solution
- Note that race conditions are solved only when we need to synthesize asynchronous systems
  - You can/should leave race conditions when you are synthesizing synchronous systems (next chapter); make sure you get this right!

METHOD1 – state encoding

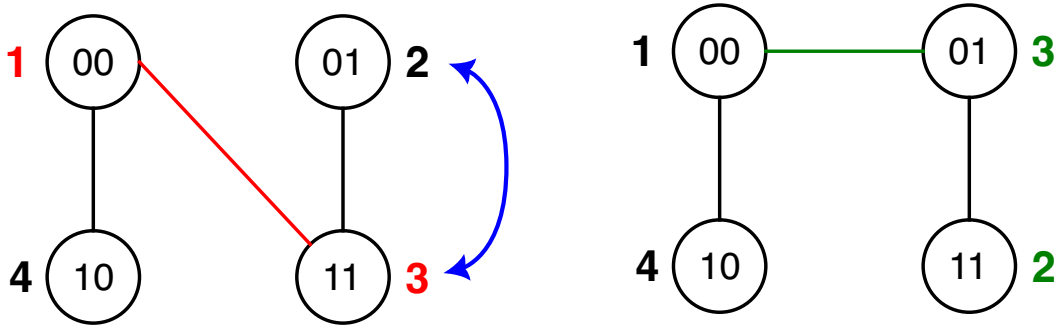
- State encoding choice is free, therefore it is maybe possible to find encoding such that encoded table is free of race conditions
  - Note that finding race free encoding is not always possible
- We want to check in a systematic way if this is possible or not
- Proposed method uses state encoding graphs to easily find encoding with no race conditions and show if such encoding exists or not; works well for small number of states
  - Take an n-cube and assign a stable state to each vertex; state transitions are shown using arcs (not necessarily oriented); this is similar to state graphs; we can omit state transition conditions since we are only interested in state-to-state connectivity
  - State assignment should respect Hamming distance: adjacent states code (vertices) have max Hamming Distance $= 1$!
  - If the above is true, then in state encoding graph: normal edge mean NO RACE conditions; diagonals indicate RACE!

METHOD1 – state encoding examples

- In the fig. below we first show an n-cube with assigned codes and then potential mapping of the states to codes (state encoding) $\rightarrow$ in this case there will be no race conditions (no diagonals); system is race free



- Let's consider another example (left below); there is a race, since there exist a transition from `00` to `11` (diagonal in state encoding graph)



- But we can swap states `2` & `3` to avoid race conditions

METHOD1 – example with state table

- For a state table and default encoding:

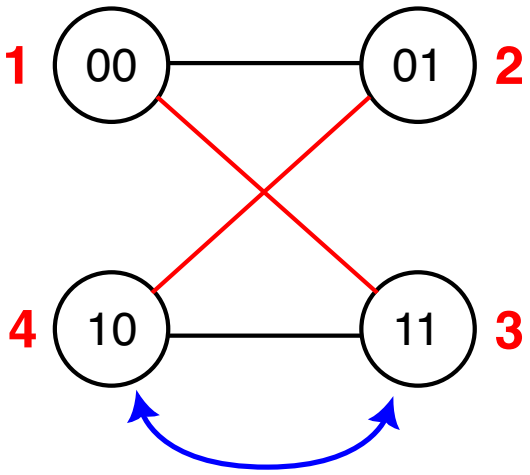
S^+ \ ab	00	01	11	10
1	**1**	2	3	2
2	**2**	**2**	4	**2**
3	1	**3**	**3**	4
4	2	3	**4**	**4**

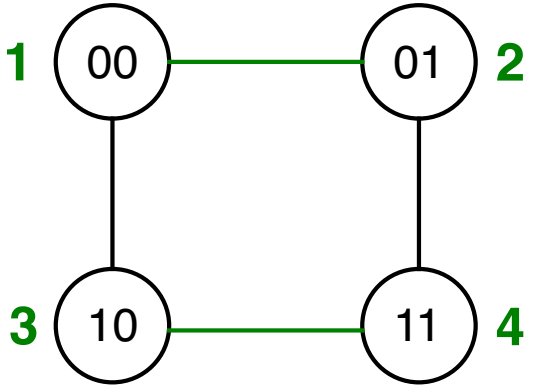
S^-

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	11	01
01	**01**	**01**	10	**01**
11	00	**11**	**11**	10
10	01	11	**10**	**10**

y_2y_1

- State encoding graphs: default encoding & swapped states 3 & 4





- One could shift state IDs for one place to create other good encodings

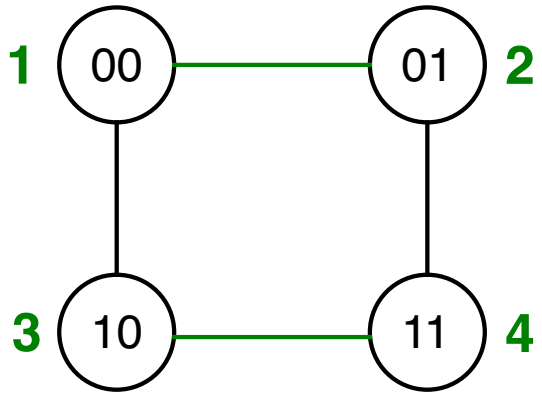
Deriving state functions

a) So for the state table:

S^+ \ ab	00	01	11	10
1	**1**	2	3	2
2	1	**2**	4	**2**
3	1	**3**	**3**	4
4	2	3	**4**	**4**

S^-

b) We adopt the encoding:



c) Encoded state table becomes:

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	10	01
01	00	**01**	11	**01**
10	00	**10**	**10**	11
11	01	10	**11**	**11**

y_2y_1

d) But the above is not a K-Map!

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	10	01
01	00	**01**	11	**01**
11	01	10	**11**	**11**
10	00	**10**	**10**	11

y_2y_1

Deriving state functions

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	10	01
01	00	**01**	11	**01**
11	01	10	**11**	**11**
10	00	**10**	**10**	11

y_2y_1

y_2y_1 \ ab	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	1	1	1
10	0	1	1	1

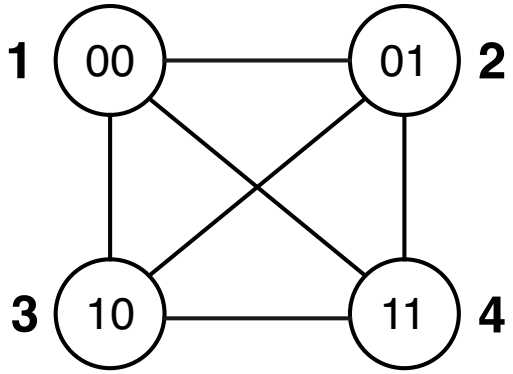
$$Y_2 = ab + y_2b + ab$$

y_2y_1 \ ab	00	01	11	10
00	0	1	0	1
01	0	1	1	1
11	1	0	1	1
10	0	0	0	1

$$Y_1 = ab' + y_1'a + y_2a'b + y_2y_1b'$$

METHOD2 – modifying transitions

- It's not always possible to find a solution using METHOD1, take the example below:



- No matter what we do, we will always have diagonals → we need something else – METHOD2 – based on the following property:

  We don't care about transitions system took,
  what counts is the source and destination stable states

- In another words three following transition sequences are equivalent since for the same source stable state we will end up in the same stable destination state:
  - ▹ **1**→2→4→**4** or **1**→3→4→**4** or **1**→2→3→5→4→**4**

METHOD2 – modifying don't cares

- Using "don't cares" as supplementary state transitions
- For the state table & encoding `1→00`, `2→01`, `3→11`, `4→10`

S^+ \ ab	00	01	11	10
1	**1**	2	3	
2		**2**	-	
3			**3**	
4				

S^-

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	11	
01		**01**	–	
11			**11**	
10				

y_2y_1

- There is a race condition (in red), but we have spare don't care just below: instead going to `11` we go to `01` and only then `11`

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	01	
01		**01**	11	
11			**11**	
10				

y_2y_1

METHOD2 – modifying existing transitions

- We can also modify the existing transitions, as long as same source states end up in the same destination states
- There is a valid (no races) transition 2→3 (11)

S^+ \ ab	00	01	11	10
1	**1**	2	3	
2		**2**	3	
3			**3**	
4				

S^-

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	11	
01		**01**	11	
11			**11**	
10				

y_2y_1

- Instead of going directly to 11 we add extra transition and borrow the existing race free transition 3, i.e., 01 and only then 11

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	01	
01		**01**	11	
11			**11**	
10				

y_2y_1

METHOD3 – adding extra state variable

- If none of the previous methods can't provide a valid solution, we can always add an extra state variable
- This is not optimal! We have spent effort to optimize the number of states to get state table with minimal number of states (minimal number of state variables); but if there is no other way, we must do that to enable functional implementation of a circuit
- By adding an extra state variable we add extra 2^n transitions (n is the initial number of state variables)
- These extra transitions are set to "don't cares" and can be used to solve race conditions as with METHOD2
- Attention! We must first solve all race conditions that can be solved using METHOD2; only those that still remain will be solved with these new transitions; if you do not do so, you will over-constrain the system and get less optimal solution in the end

Asynchronous sequential system synthesis (recap)

1. We first establish primitive state table from verbal specifications of a system; this one doesn't need to be optimal; we want to capture the exact behavior of the system; if wrong, it doesn't matter if everything else has been done correctly
   $\rightarrow$ the system will not function correctly
2. We then perform state reduction (state optimization) to reduce the number of states; possible because of possible redundancies introduced during the formalization process
3. State encoding (METHOD1) to chose the appropriate binary codes; eventually without race conditions, not always possible
4. Solution of the remaining race conditions using, METHOD2 or METHOD3; last method applied only if absolutely necessary, since it cancels the efforts deployed in step 2
5. Extraction of optimized state logic functions & output synthesis

3. Example of asynchronous implementation

Starting point – optimized state table

- Assume optimized state table below; we want to synthesize the state functions for an implementation as asynchronous logic circuit → there should be no race conditions
- We will first construct the state encoding graph to see if the METHOD1 could provide us with a solution: easiest to implement
- One (very important) thing can be spotted in the state table
  - ▹ Can you say what?

s^+ \ ab	00	01	11	10
1	**1**	2	3	4
2	1	**2**	**2**	4
3	1	4	**3**	**3**
4	1	**4**	**4**	**4**

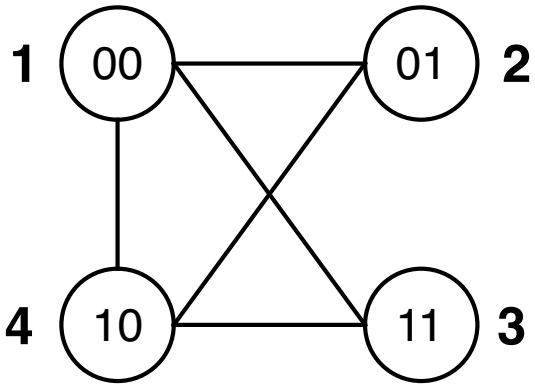
s^-

State encoding

- We start with the default encoding scheme: `1→00`, `2→01`, `3→11`, `4→10`, for which we first derive state encoding graph → no encoding without races
- `Method2` will not work – in column $ab = 11$, there is a transition with race & no other transition that can't be modified
- For chosen encoding we **must** add new state variable!
  - ▹ Using other encoding may result in implementation that would not require new variable (METHOD2 & existing transitions modification)

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	11	10
01	00	**01**	**01**	10
11	00	10	**11**	**11**
10	00	**10**	**10**	**10**

y_2y_1



State encoding graph

Adding new state

- We add an extra state variable Y_3 that introduces 16 new transitions; the size of the K-Map doubles

$Y_2 Y_1$ \ ab	00	01	11	10
00	**00**	01	11	10
01	00	**01**	**01**	10
11	00	10	**11**	**11**
10	00	**10**	**10**	**10**

$y_2 y_1$

$Y_3 Y_2 Y_1$ \ ab	00	01	11	10
000	00	01	11	10
001	00	**01**	**01**	10
011	00	10	**11**	**11**
010	00	**10**	**10**	**10**
100	-	-	-	-
101	-	-	-	-
111	-	-	-	-
110	-	-	-	-

$y_3 y_2 y_1$

Attention! K-Map on the right isn't correct!

Future is not the same as the present ...

Using new transitions to solve races

- Left table is now correct and we modify transitions to avoid races
  - From `011` to `000` is now: `111` → `110` → `100` → `000`
  - From `000` to `011` is now: `100` → `110` → `111` → `011`
  - From `001` to `010` is now: `101` → `111` → `110` → `010`

$Y_3Y_2Y_1$ \ ab	00	01	11	10
000	000	001	011	010
001	000	**001**	**001**	010
011	000	010	**011**	**011**
010	000	**010**	**010**	**010**
100	-	-	-	-
101	-	-	-	-
111	-	-	-	-
110	-	-	-	-

$y_3y_2y_1$

$Y_3Y_2Y_1$ \ ab	00	01	11	10
000	000	001	100	010
001	000	**001**	**001**	101
011	111	010	**011**	**011**
010	000	**010**	**010**	**010**
100	000	-	110	-
101	-	-	-	111
111	110	-	011	110
110	100	-	111	010

$y_3y_2y_1$

Should you use systematically new transitions?

- No!!! This over-constrains the optimization; or in the example we could modify existing transitions before using 3rd variable (2 races in blue); only one transition uses 3rd state var (green one)

$Y_3Y_2Y_1$ \ ab	00	01	11	10
000	000	001	100	010
001	000	**001**	**001**	101
011	111	010	**011**	**011**
010	000	**010**	**010**	**010**
100	000	-	110	-
101	-	-	-	111
111	110	-	011	110
110	100	-	111	010

$y_3y_2y_1$

$Y_3Y_2Y_1$ \ ab	00	01	11	10
000	000	001	100	010
001	000	**001**	**001**	000
011	001	010	**011**	**011**
010	000	**010**	**010**	**010**
100	-	-	110	-
101	-	-	-	-
111	-	-	011	-
110	-	-	111	-

$y_3y_2y_1$

K-Maps optimization for state functions – Y_1

y_2y_1 \ ab	00	01	11	10
00	0	1	0	0
01	0	1	1	0
11	1	0	1	1
10	0	0	0	0

$y_3 = 0$

y_2y_1 \ ab	00	01	11	10
00	-	-	0	-
01	-	-	-	-
11	-	-	1	-
10	-	-	1	-

$y_3 = 1$

$$Y_1 = y_2' a' b + y_1 ab + y_2 y_1 b' + y_3 y_2$$

K-Maps optimization for state functions – Y_2

y_2y_1 \ ab	00	01	11	10
00	0	0	0	1
01	0	0	0	0
11	0	1	1	1
10	0	1	1	1

$y_3 = 0$

y_2y_1 \ ab	00	01	11	10
00	-	-	1	-
01	-	-	-	-
11	-	-	1	-
10	-	-	1	-

$y_3 = 1$

$$Y_2 = y_2 b + y_2 a + y_3 a + y_1' a b'$$

K-Maps optimization for state functions – Y_3

y_2y_1 \ ab	00	01	11	10
00	0	0	1	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

$y_3 = 0$

y_2y_1 \ ab	00	01	11	10
00	-	-	1	-
01	-	-	-	-
11	-	-	0	-
10	-	-	1	-

$y_3 = 1$

$$Y_3 = y_2' y_1' ab + y_3 y_1'$$

Alternative initial encoding

- If we chose the default encoding we need to add another variable
- But how about alternative encoding?
  - For example: `1` $\rightarrow$ `00`, `2` $\rightarrow$ `01`, `4` $\rightarrow$ `11`, & `3` $\rightarrow$ `10`
- We will still have two races (right table, marked in red) but they could be solved with METHOD2 & existing transitions without adding an extra variable $\rightarrow$ simpler, thus preferable solution
  - Note that in the K-Map, lines `11` & `10` have been swaped

S^+ \ ab	00	01	11	10
1	**1**	2	3	4
2	1	**2**	**2**	4
3	1	4	**3**	**3**
4	1	**4**	**4**	**4**

S^-

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	10	11
01	00	**01**	**01**	11
11	00	**11**	**11**	**11**
10	00	11	**10**	**10**

y_2y_1

Alternative initial encoding – solution

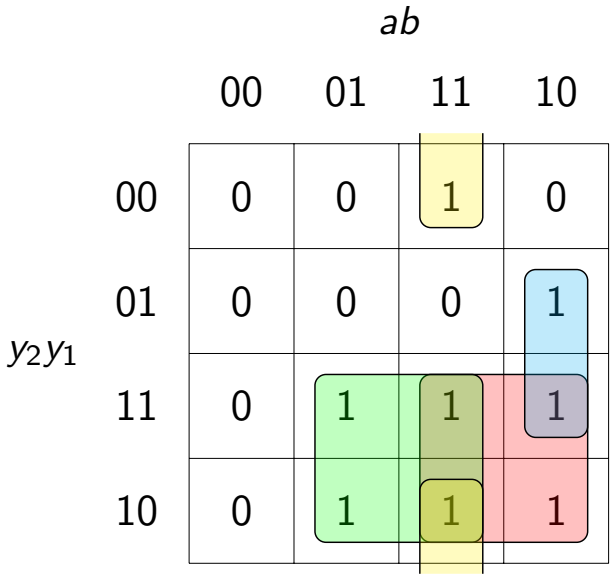
Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	10	11
01	00	**01**	**01**	11
11	00	**11**	**11**	**11**
10	00	11	**10**	**10**

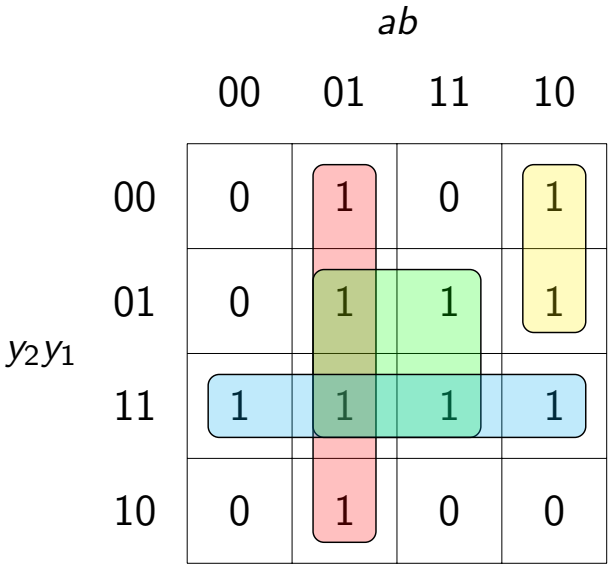
y_2y_1

$\Rightarrow$

Y_2Y_1 \ ab	00	01	11	10
00	**00**	01	10	01
01	00	**01**	**01**	11
11	01	**11**	**11**	**11**
10	00	11	**10**	**10**

y_2y_1





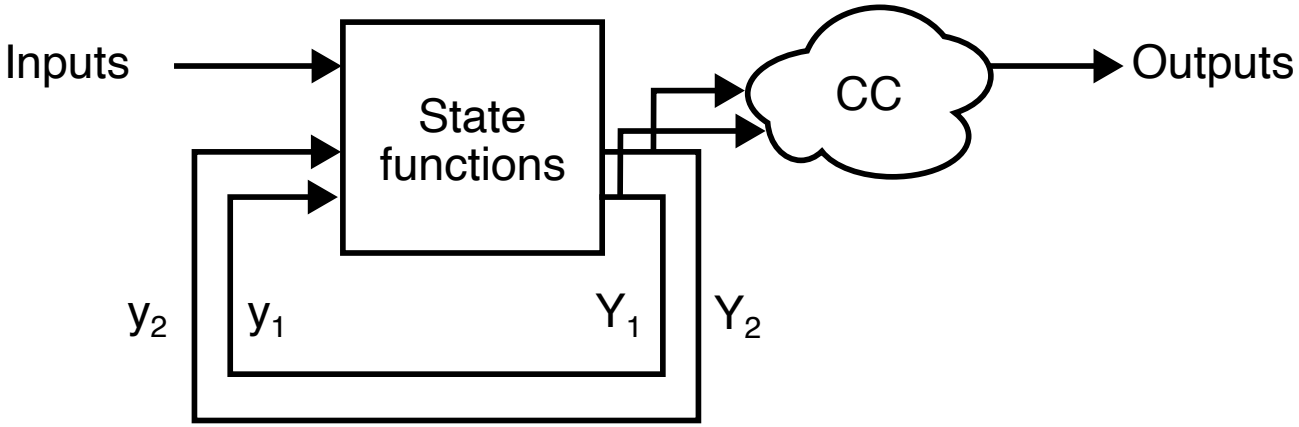
$$Y_2 = y_2a + y_2b + y_1'ab + y_1ab'$$

$$Y_1 = a'b + y_1b + y_2'ab' + y_2y_1$$

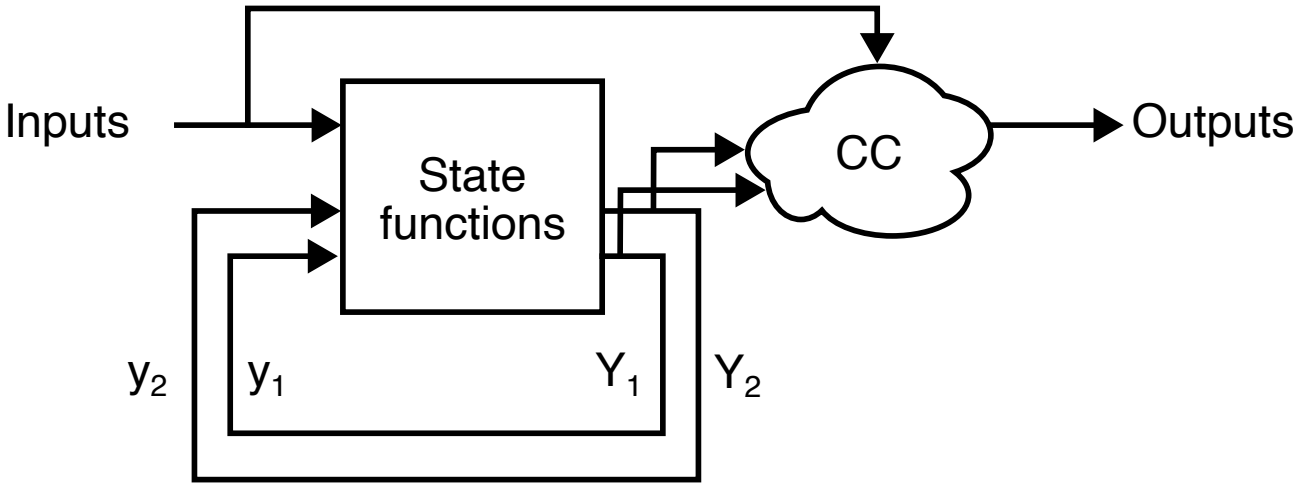
4. Synthesis of Moore & Mealy machines including output

Moore & Mealy sequential machines

- Moore – output is a combinatorial function of state variables only



- Mealy – output is function of state variables and system input



Moore machine – synthesis of output(s)

- Output is synthesized using state decoder (combinatorial circuit): sum of products (states) for which the output is set to '1'

				ab	
Y_2Y_1	00	01	11	10	Z
00	**00**	01	01	–	0
01	00	**01**	11	–	1
11	**11**	01	**11**	10	0
10	–	**10**	11	10	1

y_2y_1

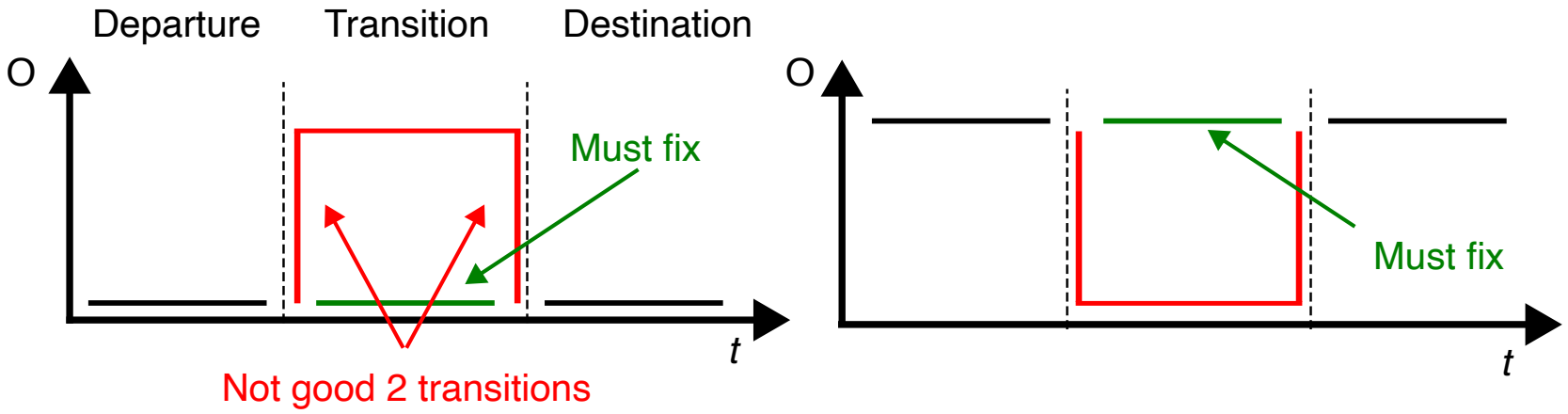
- Then the output is simply a Boolean function of states:

$$Z = f(y_i) = y_2'y_1 + y_2y_1'$$

- Not a good way to synthesize the output!
  - Look at transitions **00** → 01 → 11 → **11** in the table above, for which we have the following output change '**0**' → '0' → '1' → '**0**'; for transition 11 there is a glitch that we should avoid!

Output transitions – single transition case (1/2)

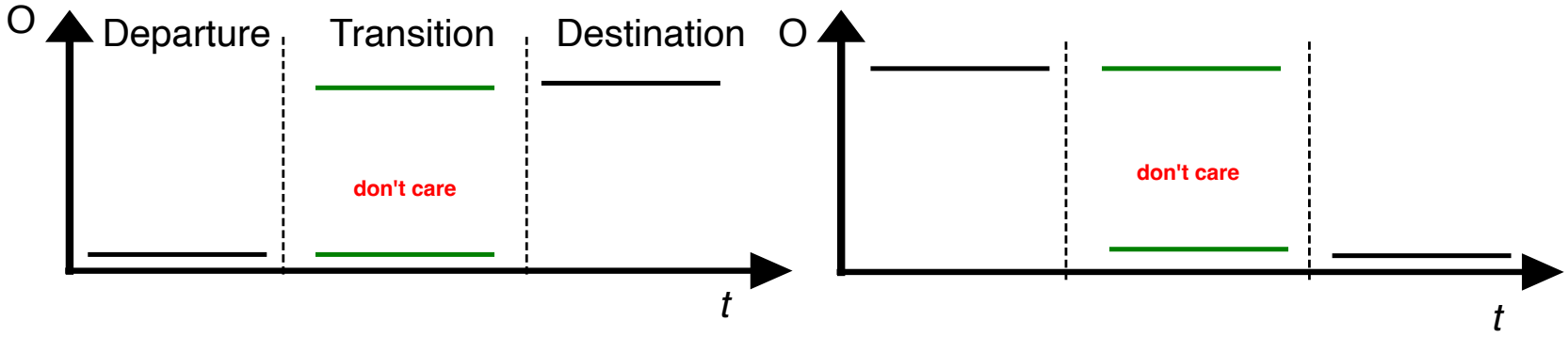
- We need to fix the value of the output during the transition, so that only one transition occurs from '0' to '1' or '1' to '0'
  - The chosen value will depend on the output of the departure and destination states
- If the output of the departure state & destination state **are the same** the output value should remain the same during the transition to avoid more than one change



- On the left we must fix '0' and on the right we must fix '1'

Output transitions – single transition case (2/2)

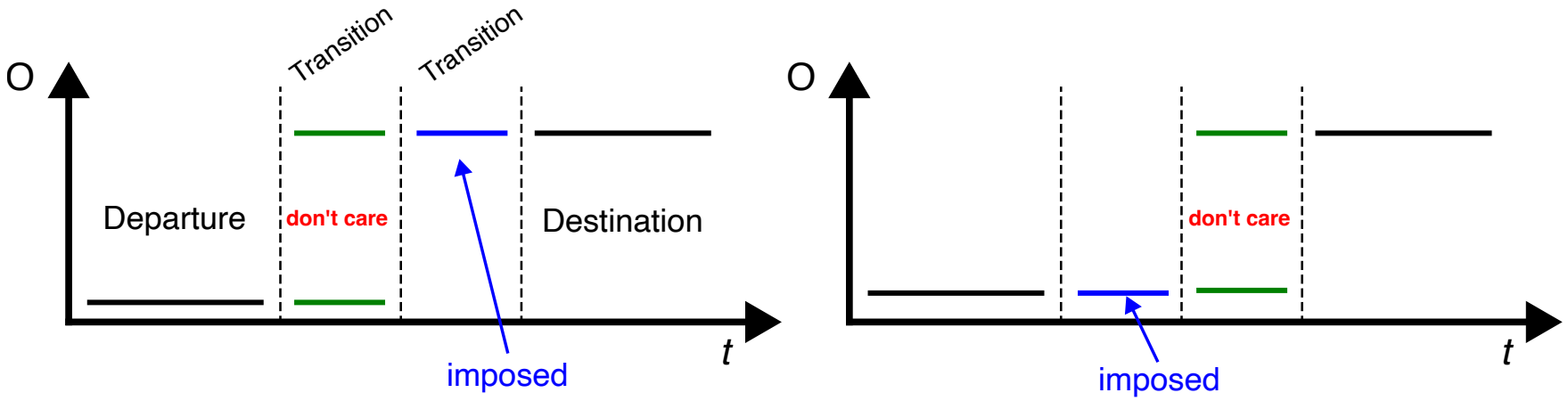
- If the output for departure state & destination state **are different** either '0' or '1' could work, the output will change only once
- The transition can be set to don't care!



- Looking at the left figure (similarly for the right), if during synthesis we chose '0' for don't care, the output transition will occur later than if we picked '1'; but the absolute time when the transition occurs is of no importance!

Output transitions – case of multiple transitions

- When solving race conditions it is possible that we introduced multiple transitions between two stable states (METHOD2)
- How to fix the output in such cases?
  - Same rule is applied: only one transition at the output is allowed!



- Only one don't care can be used, other transitions must be fixed
  - In the example on the left second transition is imposed, don't care can be placed for a first transition; on the right first transition is imposed, second could be don't care

Output transitions – shared transitions

- Attention! Transitions between stable states can be shared, especially after we applied METHOD2 to solve race conditions
- In the example we have **2** $\rightarrow$ 3 $\rightarrow$ **3**, but also **1** $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ **3**
- Transition 3 is shared and the state transition **1** $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ **3** will fix the value for the output at 3, and not **2** $\rightarrow$ 3 $\rightarrow$ **3**

s^+ \ ab	00	01	11	10	Z
1	**1**	2	2		0
2		**2**	3		1
3			**3**		0
4					

s^-

z \ ab	00	01	11	10
1	**0**	-	0	
2		**1**	0	
3			**0**	
4				

s^-

Moore machine – example of output synthesis

Output K-Map based on a single transition rule:

S^+ \ ab	00	01	11	10	Z
1	**1**	2	2	-	0
2	1	**2**	3	-	1
3	**3**	2	**3**	4	0
4	-	**4**	3	**4**	1

S^-

$\Downarrow$

S^+ \ ab	00	01	11	10
1	**0**	-	0	-
2	-	**1**	0	-
3	**0**	-	**0**	-
4	-	**1**	-	**1**

S^-

$\Rightarrow$

y_2y_1 \ ab	00	01	11	10
00	0	-	0	-
01	-	1	0	-
11	0	-	0	-
10	-	1	-	1

$$Z = a'b + ab'$$

or

$$Z = a'b + y_2y_1'$$

Mealy machine as opposed to Moore

- In Moore machines the output is a function of system states only; for this reason during state optimization we could not fuse states with different outputs, so we have something like $Z = f(y_i)$
- But to avoid glitches at system output we limit the number of transitions to only one! (1 change from 0 to 1 and inversely)
- This forced us to explicitly set the output during transitions, which in turn might add inputs in the output logic equation and $Z = f(y_i, a, b)$
  - ▹ Is this then a Mealy machine? $\rightarrow$ NO!
- Difference between the Moore & Mealy appear much earlier in the synthesis process – during state optimization
- Because in Mealy machines we can use inputs to calculate the output, we can differentiate stable states in a single state table line, so that they could have different outputs!

State fusion example – Moore & Mealy

- For the system below: states 1 & 4 can't be fused if we are implementing Moore machine; this is because we can't differentiate the output for the fused state **4** (should be '1') from the output of the state **1** (should be '0')

				ab	
S^+	00	01	11	10	Z
1	**1**	3	2	-	0
2					1
3					0
4	1	3	2	**4**	1
S^-					

- In Mealy machine fusing 1 and 4 is possible!!!
- Important consequence – in Mealy machines we could possibly merge more states; if this is the case, the resulting system could be simpler since less logic is required for the implementation

Mealy machine – output

- Output is a function of inputs too, so same stable states can have different outputs (not possible in Moore machines, the output is function of state only)
- Rather than a single output column (one per state) as for Moore machines, in Mealy machines there will be one output column per input combination; here the output is written next to the stable state ID separated with a slash
- Below there is a difference between two stable states **1**: one with output set to '1' for $ab = 00$ & '0' for $ab = 10$

	ab			
s^+	00	01	11	10
1	**1/1**		2	**1/0**
2			**2/0**	
s^-				

Mealy machine: states that can't fuse

- Not all states with different outputs can be merged
- Two stable states having different outputs and being stable for the same combination of inputs can't be fused
  - ▹ I.e., stables states in the same column with different outputs
- In the example below we can not make a difference between states **1** and **4** if they were merged!

				ab	
s^+	00	01	11	10	Z
1	**1**	-	2	3	1
2	1	-	**2**	3	0
3	1	-	2	**3**	0
4	**4**	-	2	3	0
s^-					

Mealy machine – output synthesis

- Same idea is applied as for Moore machine:
  - Single output value transition (from '0 ' to '1' or '1' to '0') is allowed between two stable states no matter how many transitions we have
- Attention! Things can be a bit more complicated in case of Mealy machines, since we can have two (or more) outputs that might be different for different stable states of the same present (same line in the state table, see next slide)
  - Mealy output could therefore be more constrained than for Moore machine, and could thus have more complex final expressions
- Two cases (as previously):
  - Single transition between departure and destination states
  - Multiple transitions between departure and destination states, that could be eventually shared

Machine de Mealy – output synthesis 1/2

- Transition 2 from state 1 when input $ab = 11$ can be taken:
  a. from state 1 when $ab = 00$, $Z = 1$ or
  b. from state 1 when $ab = 10$, $Z = 0$

s^+/Z \ ab	00	01	11	10
1	**1/1**		2	**1/0**
2			**2/0**	
3				
4				

s^-

Z \ ab	00	01	11	10
1	**1**		0	**0**
2			**0**	
3				
4				

s^-

It is the transition from $ab = 10$ with $Z = 0$ that will force the output value for transition 2 to be '0'

Machine de Mealy – output synthesis 2/2

- Attention shared transitions – in the example transition 3 for $ab = 11$ is used to move from state 1 to 3, but also from 2 to 3
- We need to fix the output for transition from 2 to 3 to '0' first!
- Transition 2 must be fixed later, accordingly to what has been set for 3; you must find the right output assignment sequence
  - Output for transition 3 being set to '0' and both outputs for state 1 being 1 $\rightarrow$ we can set output for transition 2 as don't care

s^+/z \ ab	00	01	11	10
1	**1/1**		2	**1/1**
2		**2/0**	3	
3			**3/0**	
4				

s^-

z \ ab	00	01	11	10
1	**1**		-	**1**
2		**0**	0	
3			**0**	
4				

s^-

5. Synthesis of Moore & Mealy machines

Moore machine

- System behaviour is specified using the primitive state table below
- State optimisation & synthesis done for Moore & Mealy machines
- Both solutions are then compared in terms of logic complexity
  - ▹ A priori Mealy machine could have less states due to possibly better state optimisation (we can merge states that have different outputs)

	ab				
S^+	00	01	11	10	Z
1	**1**	2	-	3	0
2	1	**2**	4	-	1
3	1	-	4	**3**	0
4	-	5	**4**	-	1
5	6	**5**	4	-	0
6	**6**	5	-	3	1
S^-					

A. Moore machine – state fusions, 1st pass

- We build equivalence conditions table – states with different outputs (those that can't be fused) are marked with X; otherwise we write fusion conditions, or OK if there are no conditions

S^+ \ ab	00	01	11	10	z
1	**1**	2	-	3	0
2	1	**2**	4	-	1
3	1	-	4	**3**	0
4	-	5	**4**	-	1
5	6	**5**	4	-	0
6	**6**	5	-	3	1

S^-

2	X				
3	OK	X			
4	X	2-5	X		
5	1-6 2-5	X	1-6	X	
6	X	1-6 2-5	X	OK	X
	1	2	3	4	5

A. Moore machine – state fusions, 2nd pass

Conditions 1-6 and 2-5 are examined → they can not be fused!

2	X				
3	OK	X			
4	X	2-5	X		
5	1-6 2-5	X	1-6	X	
6	X	1-6 2-5	X	OK	X
	1	2	3	4	5

2	X				
3	OK	X			
4	X	2-5	X		
5	1-6 2-5	X	1-6	X	
6	X	1-6 2-5	X	OK	X
	1	2	3	4	5

Fusion graphs are very simple, we can only do `1-3` and `4-6`

A. Moore machine – initial and fused table

1 and 3 $\rightarrow$ 1 2 $\rightarrow$ 2
4 and 6 $\rightarrow$ 3 5 $\rightarrow$ 4

S^+ \ ab	00	01	11	10	z
1	**1**	2	-	3	0
2	1	**2**	4	-	1
3	1	-	4	**3**	0
4	-	5	**4**	-	1
5	6	**5**	4	-	0
6	**6**	5	-	3	1

S^-

S^+ \ ab	00	01	11	10	z
1	**1**	2	-	**1**	0
2	1	**2**	3	-	1
3	**3**	4	**3**	1	1
4	3	**4**	3	-	0

S^-

A. Moore machine – state encoding

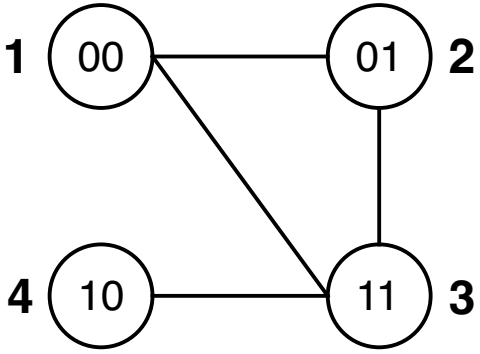
- State encoding graph: we will always have one race condition

S^+ \ ab	00	01	11	10	z
1	**1**	2	-	**1**	0
2	1	**2**	3	-	1
3	**3**	4	**3**	1	1
4	3	**4**	3	-	0

S^-

S^+ \ ab	00	01	11	10	z
00	**00**	01	-	**00**	0
01	00	**01**	11	-	1
11	**11**	10	**11**	00	1
10	11	**10**	11	-	0

S^-



The encoding scheme has one race (marked in red)

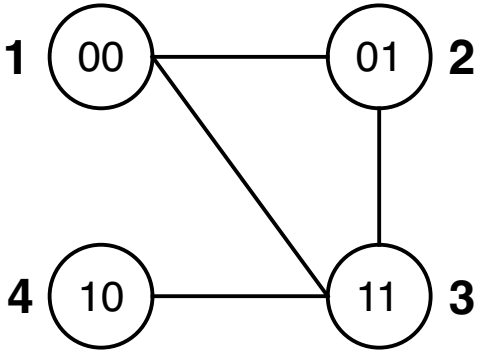
A. Moore machine – solving race conditions

Y_2Y_1 \ ab	00	01	11	10	Z
00	**00**	01	-	**00**	0
01	00	**01**	11	-	1
11	**11**	10	**11**	00	1
10	11	**10**	11	-	0

y_2y_1

Y_2Y_1 \ ab	00	01	11	10	Z
00	**00**	01	-	**00**	0
01	00	**01**	11	00	1
11	**11**	10	**11**	01	1
10	11	**10**	11	-	0

y_2y_1



Race can be easily solved using METHOD2

$11 \rightarrow 00$ is replaced with

$11 \rightarrow 01 \rightarrow 00$

A. Moore machine – state variables

Y_2Y_1 \ ab	00	01	11	10	Z
00	**00**	01	-	**00**	0
01	00	**01**	11	00	1
11	**11**	10	**11**	01	1
10	11	**10**	11	-	0

y_2y_1 \ ab	00	01	11	10
00	0	0	-	0
01	0	0	1	0
11	1	1	1	0
10	1	1	1	-

$$Y_2 = y_2a' + ab$$

y_2y_1 \ ab	00	01	11	10
00	0	1	0	0
01	0	1	1	0
11	1	0	1	1
10	1	0	1	-

$$Y_1 = y_2'b + y_2b' + ab$$

A. Moore machine – output synthesis

a) Initial table with output:

Y_2Y_1 \ ab	00	01	11	10	Z
00	**00**	01	-	**00**	0
01	00	**01**	11	00	1
11	**11**	10	**11**	01	1
10	11	**10**	11	-	0

y_2y_1

b) Outputs for stable states:

Z \ ab	00	01	11	10
00	**0**		-	**0**
01		**1**		
11	**1**		**1**	
10		**0**		-

y_2y_1

c) Outputs with transitions:

Z \ ab	00	01	11	10
00	**0**	-	-	**0**
01	-	**1**	1	-
11	**1**	-	**1**	1
10	-	**0**	-	-

y_2y_1

d) K-Map:

y_2y_1 \ ab	00	01	11	10
00	0	-	-	0
01	-	1	1	-
11	1	-	1	1
10	-	0	-	-

$Z = y_1$

B. Mealy machine – state fusions, 1st pass

- New state equivalence table in which we can merge states with different outputs: all except 1 & 6 and 2 & 5 (stable states with different outputs for same input combination)

S^+ \ ab	00	01	11	10	z
1	**1**	2	-	3	0
2	1	**2**	4	-	1
3	1	-	4	**3**	0
4	-	5	**4**	-	1
5	6	**5**	4	-	0
6	**6**	5	-	3	1

S^-

2	OK				
3	OK	OK			
4	2-5	2-5	OK		
5	1-6 2-5	X	1-6	OK	
6	X	1-6 2-5	1-6	OK	OK
	1	2	3	4	5

B. Mealy machine – state fusions, 2nd pass

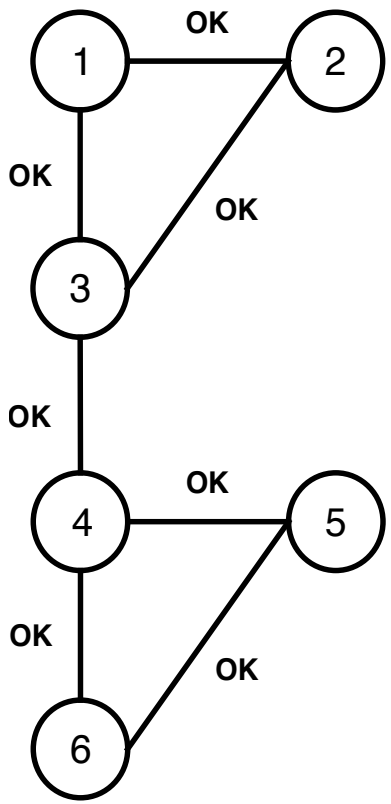
All cells that contain `1` & `6` and `2` & `5` are invalidated (here all cells with conditions will contain at least one of the above!)

2	OK				
3	OK	OK			
4	2-5	2-5	OK		
5	1-6 2-5	X	1-6	OK	
6	X	1-6 2-5	1-6	OK	OK
	1	2	3	4	5

2	OK				
3	OK	OK			
4	2-5	2-5	OK		
5	1-6 2-5	X	1-6	OK	
6	X	1-6 2-5	1-6	OK	OK
	1	2	3	4	5

B. Mealy machine – state fusion graphs

2	OK				
3	OK	OK			
4	X	X	OK		
5	X	X	X	OK	
6	X	X	X	OK	OK
	1	2	3	4	5



- Clear choice: `1` & `2` & `3` → `1` and `4` & `5` & `6` → `2`
- Alternative would be `1` & `2` → `1`; `3` & `4` → `2`; `5` & `6` → `3`, but this would clearly be an inferior choice (3 instead of 2 final states)
- Be careful when choosing what to fuse

B. Mealy machine – final fused table with outputs

1 & 2 & 3 → 1
4 & 5 & 6 → 2

S^+ \ ab	00	01	11	10	z
1	**1**	2	-	3	0
2	1	**2**	4	-	1
3	1	-	4	**3**	0
4	-	5	**4**	-	1
5	6	**5**	4	-	0
6	**6**	5	-	3	1

S^-

S^+ \ ab	00	01	11	10
1	**1/0**	**1/1**	2	**1/0**
2	**2/1**	**2/0**	**2/1**	1

S^-

B. Mealy machine – K-Maps & optimized functions

S^+ \ ab	00	01	11	10
1	**1/0**	**1/1**	2	**1/0**
2	**2/1**	**2/0**	**2/1**	1

S^-

y_1 \ ab	00	01	11	10
0	0	0	1	0
1	1	1	1	0

y_1 \ ab	00	01	11	10
0	0	1	1	0
1	1	0	1	0

$$Y_1 = y_1 a' + ab$$

$$Z = ab + y_1' b + y_1 a' b'$$

Moore vs. Mealy – side by side comparison

- **Moore machine** – state functions & output:

$$\begin{aligned} Y_2 &= y_2 a' + ab \\ Y_1 &= y_2' b + y_2 b' + ab \\ Z &= y_1 \end{aligned}$$

- **Mealy machine** – state functions & output:

$$\begin{aligned} Y_1 &= y_1 a' + ab \\ Z &= ab + y_1' b + y_1 a' b' \end{aligned}$$

- Mealy uses less circuits (2 instead of 3), but has slightly more complicated output function; we can not say in advance which one is better! (i.e., Mealy is not systematically better)